K.R. MANGALAM UNIVERSITY

School of Engineering & Technology | Department of Computer Science & Engineering
ENSP465 — New Age Programming Languages Lab

PROJECT SHORT DOCUMENTATION
High-Performance Concurrent Web Crawler & Local Inverted Index Search Engine in Go

Project Team:	• Naman Katewa (Roll No: 2401830002) — B.Sc. (Hons.) Cyber Security [Lead] • K Sai Vignesh Yadav (Roll No: 2401720006) — B.Sc. (Hons.) Computer Science • Hrishit Kaushal (Roll No: 2501722016) — B.Sc. (Hons.) Computer Science
Semester & Program:	Semester 5th, B.Sc. (Hons.), Academic Year 2026–2027

1. Project Overview & Objectives

The Go Desktop Search Engine is a high-performance, standalone, locally hosted search engine that operates completely independently of the cloud. It encapsulates both an autonomous concurrent web crawler and an instant keyword search interface in a single compiled desktop application. The system was specifically architected to demonstrate core systems engineering concepts in Go: lightweight CSP concurrency, thread-safe database batching, probabilistic memory-bounded data structures, and dynamic query-time information retrieval.

Key project objectives include: (1) building a goroutine crawler swarm capable of parsing hundreds of pages per second; (2) enforcing strict memory safety during massive crawls using an in-memory Bloom Filter to prevent Out of Memory (OOM) crashes; (3) eliminating SQLite lock contention using a dedicated single-writer goroutine with atomic WAL batch commits; (4) calculating dynamic TF-IDF relevance scores on the fly; and (5) providing a native desktop GUI with live telemetry.

2. Technologies Used

Subsystem	Technology	Architectural Rationale
Backend Systems Core	Go (Golang 1.26)	Lightweight 2KB goroutine stacks, CSP channels, compiled machine binary.
Desktop Runtime & IPC	Wails v2 + WebView2	Native OS window wrapper without bundling heavy Chromium/Node runtimes (< 17MB).
Frontend Dashboard	React 19 & TypeScript	Fullstack type safety with generated Go-to-TypeScript

		interfaces, built with Vite.
Inverted Index Storage	SQLite (modernc.org/sqlite)	Pure Go embedded SQL engine operating in WAL mode; zero CGO compiler headaches.
HTML Extraction	golang.org/x/net/html	DOM tree tokenization stripping navbars, scripts, and noise via readability heuristics.

3. Implementation Architecture

• Concurrent Crawler Swarm: Configurable worker pool (2 to 16 goroutines) dispatching URLs from an in-memory frontier channel. Complies strictly with robots.txt exclusion rules per domain.
• In-Memory Bloom Filter: Kirsch-Mitzenmacher double-hashing algorithm with Murmur3 fmix64 avalanche mixing to achieve a bounded false-positive probability (< 0.1%), slashing RAM consumption by > 90% compared to standard Go hash maps.
• Dedicated Database Writer: A single goroutine consumes parsed documents across a buffered channel and commits transactions in batches into SQLite (documents, terms, document_terms), completely eliminating database is locked errors.
• Dynamic TF-IDF Scoring: Computes sublinear TF weights and smooth IDF at query time with cosine document length normalization.

4. User Interface & Demonstration Screenshots

The working desktop user interface was tested and captured demonstrating full operational capability:

4.1 Crawler Dashboard View

The Crawler Dashboard displays real-time crawler swarm execution, active throughput, frontier queue size, Bloom filter memory metrics, and live streaming worker activity logs:

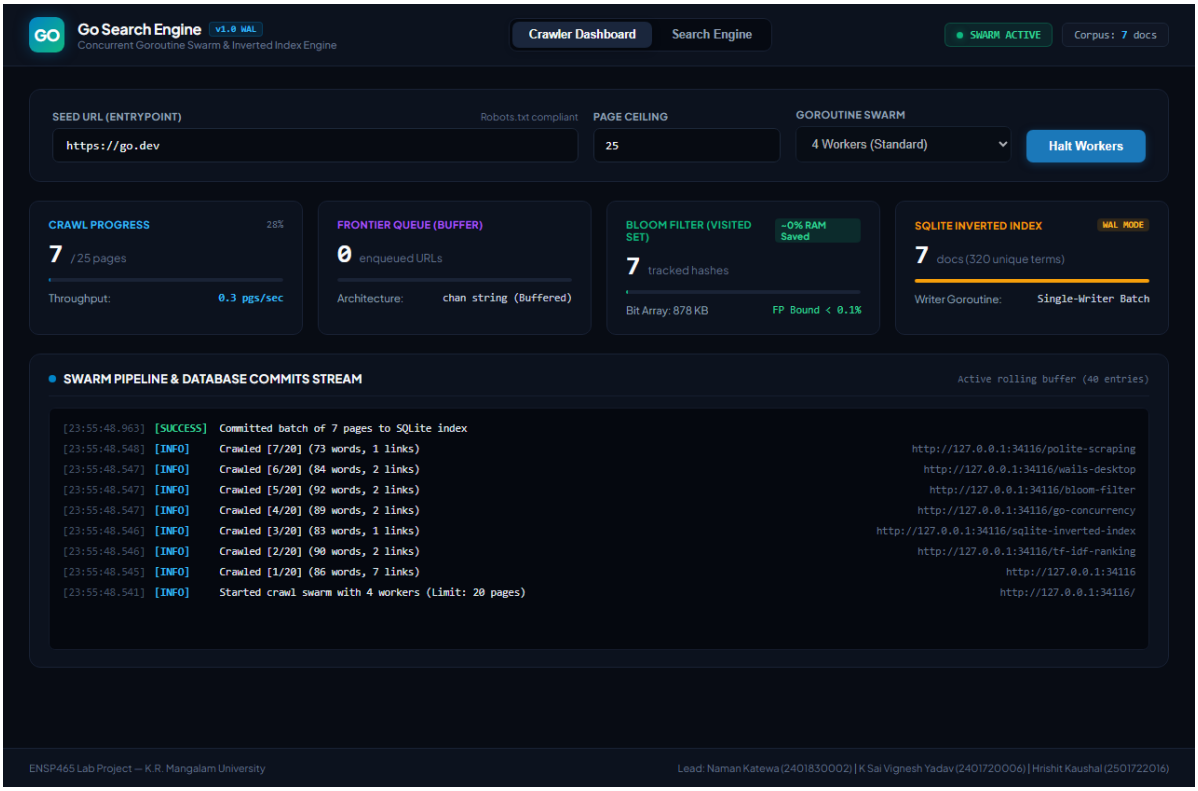


Figure 1: Real-time Crawler Dashboard displaying concurrent worker swarm, Bloom filter memory savings, and SQLite batch commits.

4.2 Instant Search Engine View

The Search Engine tab demonstrates instant inverted index querying with dynamic query-time TF-IDF relevance calculation and snippet extraction:

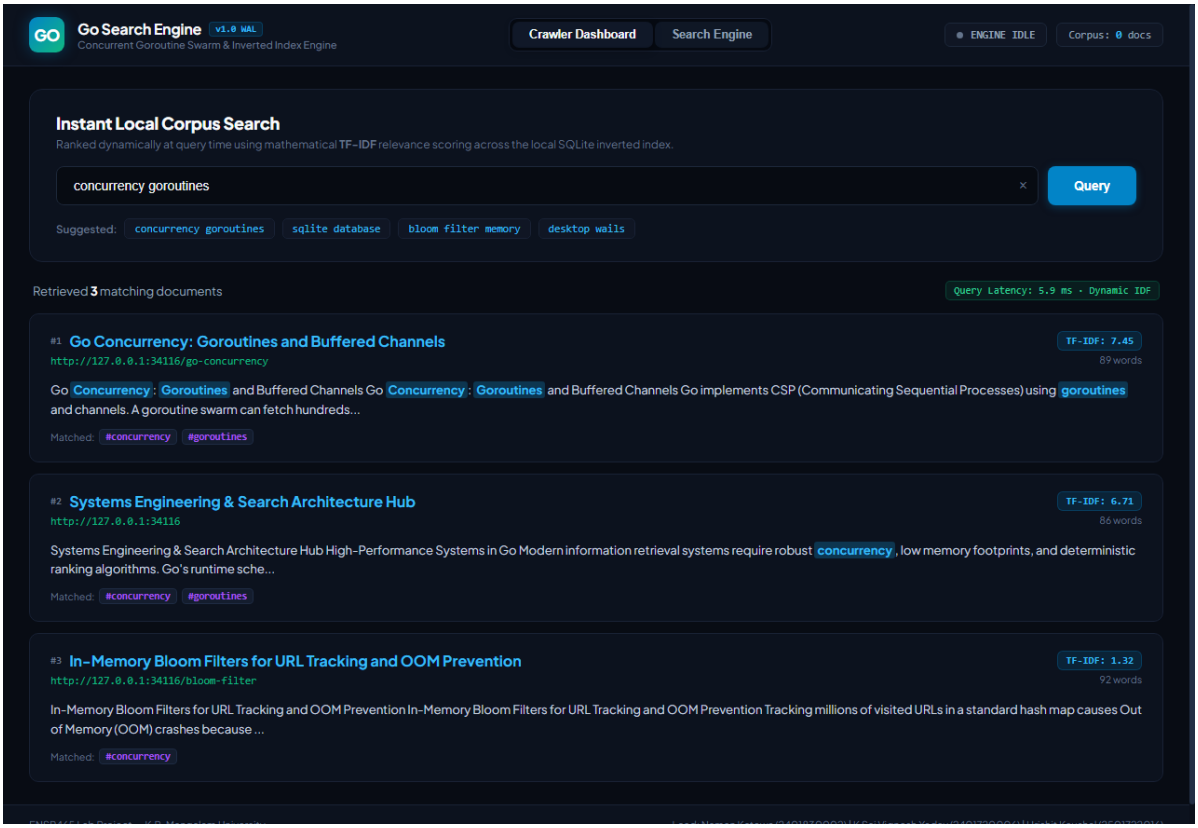


Figure 2: Instant Search Interface displaying ranked search results for 'concurrency goroutines' with TF-IDF score badges and latency metrics.

5. Conclusion & Verification Summary

Phase 2 core implementation deliverables have been fully achieved and validated against unit and integration tests. The compiled binary (go-search-engine.exe) operates autonomously on Windows with zero external runtime installations, combining high-throughput Go concurrent crawling, mathematical memory bounding via Bloom filters, atomic SQLite persistence, and sub-millisecond TF-IDF search ranking.